SLIIT | FACULTY OF COMPUTING | DEPARTMENT OF SOFTWARE ENGINEERING

SE3090 – Software Engineering Frameworks

Lecture 05

Agentic AI Fundamentals – Part 1

Year 3 • Semester 1 • 4 Credits

Lecturer: Lasal Hettiarachchi | lasal.h.visiting@sliit.lk | Course web: courseweb.sliit.lk

Today's Roadmap – 3 Hours

Hour 1	**Frameworks & Fundamentals** Introduction to Agentic AI | Frameworks | Fundamental knowledge about Models
Hour 2	**MODEL + TOOLS + LOOP + GOALS** Effective tool usage in Agentic AI
Hour 3	**Composition** Quick intro to the LABS

Short breaks after each hour • Questions welcome at any point

PART 1 OF 4

Fundamentals Of LLMs

- How LLMs work
- Limitations of LLM workflows
- What is agentic AI
- Agentic AI spectrum



Agent vs Model

Prompt

"Find me a flight to Singapore next Friday under $400 and hold a seat."

smart_toy

Traditional LLM Model

Provides information and static advice, but cannot take action.

"Here are some tips for finding cheap flights..."

task_alt

AI Agent

Executes tasks, calls APIs, and performs real-world actions to achieve goals.

"Held seat 34C on TR 561, Confirmation #8842"

Small pieces of words combine to form sentences



Tokens

Tokens are the foundational units of text processing in NLP, created by breaking down raw text into smaller components like words, subwords, or even characters.



Embeddings

Embeddings are the result of training vectorized tokens to capture semantic relationships between words. Unlike raw vectors, embeddings position similar words (e.g., "king" and "queen") closer in vector space, allowing models to understand context and meaning.

https://medium.com/@saschametzger/what-are-tokens-vectors-and-embeddings-how-do-you-create-them-e2a3e698e037

Small pieces of words combine to form sentences

Tokens

Example sentence: **“The cat eats fish.”**

1. Tokens split the sentence

["The", "cat", "eats", "fish"]

Each token receives an ID:

The → 10 | eats → 40
cat → 25 | fish → 61

However, the number **25** does not explain what “cat” means.

Embeddings

The **model** converts each **token ID** into a **vector:**

cat → represents an animal
eats → represents an action
fish → represents food in this context

The embedding for **“cat”** will be more similar to **“dog”** than to **“car.”**

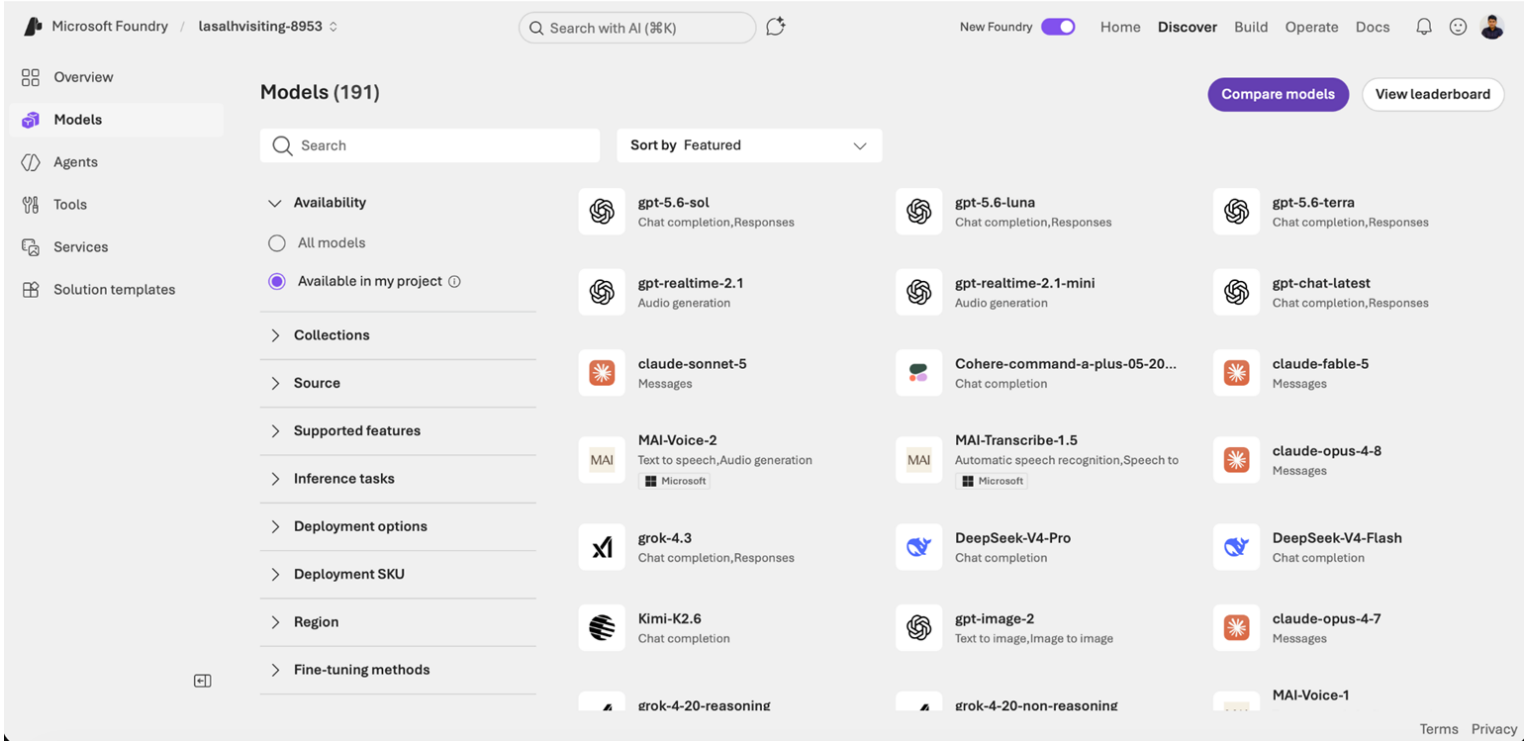
- **Tokens** help the model separate and identify the words.
- **Embeddings** help the model understand their meanings and relationships.

What is an LLM, really?

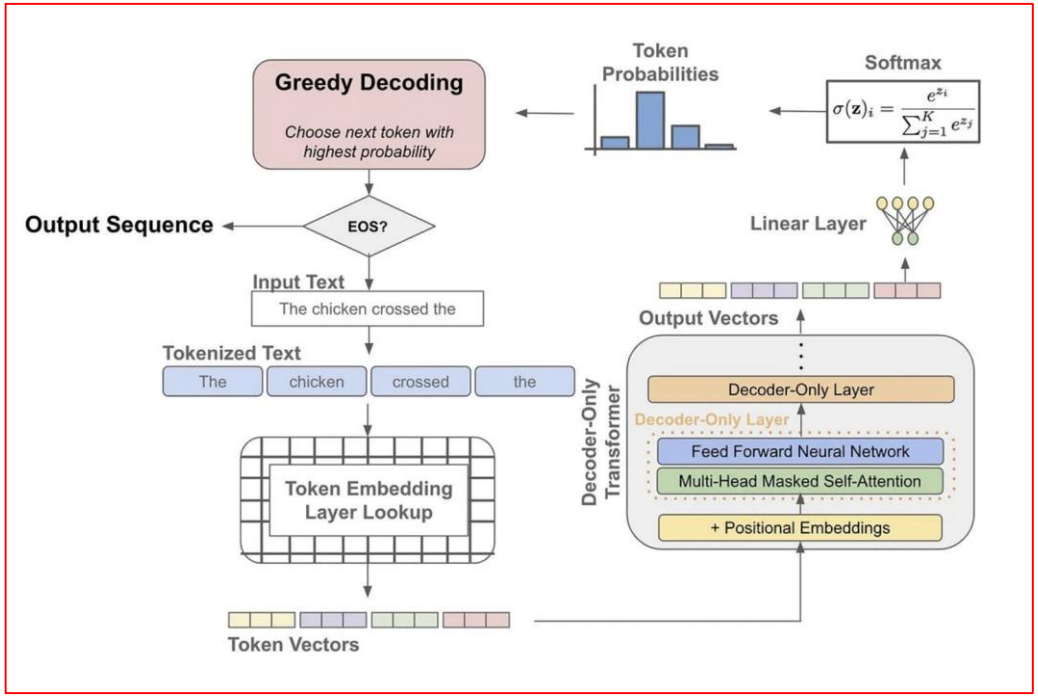
Definition: A neural network trained to predict the next token given the previous tokens. Usually the first set of seed text (user prompt) and the system message is provided.

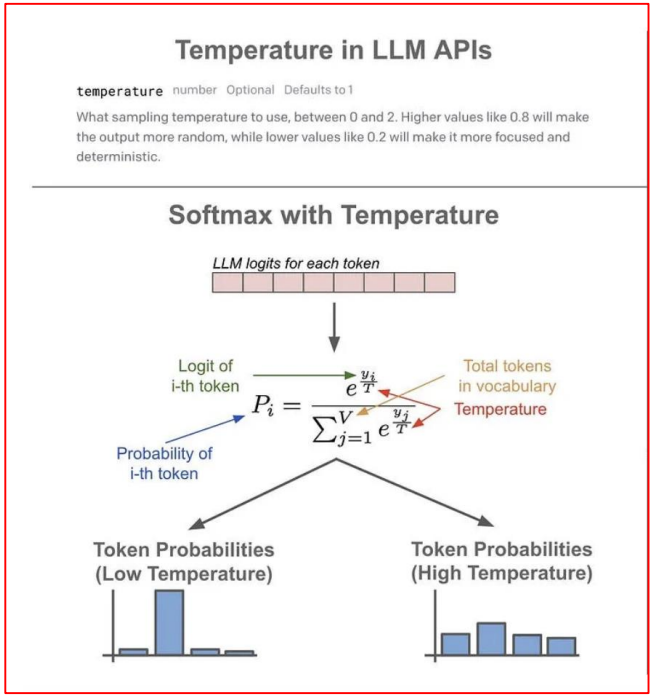
- "Token" ≈ a chunk of text (~¾ of a word). Text is split into tokens first.
- Trained in two broad stages (high level):
  - **Pre-training** — read enormous amounts of text → learn language & world patterns.
  - **Alignment / instruction-tuning** — learn to *follow instructions* and be helpful.
- At inference it is, fundamentally, a **function**: `tokens in → next token out`, applied repeatedly to generate text.

Model Catalog



What is an LLM, really?





What is an LLM, really?

“The chicken crossed the… ? ”

1. The sentence is first divided into **tokens**.
2. Tokens are converted into **embeddings** so the model can process their meanings.
3. **Transformer layers** examine the context and relationships between tokens.
4. The model gives every possible next tokens raw scores called **logits**.
5. A **higher logit** means the ***token is more suitable***.
6. **Softmax** converts the logits into **probabilities** that total **100%**.. (road → 70% , street → 20% , river → 10%)
7. The model selects **“road,”** adds it to the sentence, and repeats the process.

Simple flow: **Text → Tokens → Embeddings → Probabilities → Select next token → Repeat**

From hidden state to a probability

At each step the network emits a vector of logits — one raw score zi for every token i in the vocabulary V.

- **Hidden state** is the model's internal understanding of the text so far.
- Logits become a probability distribution via the **softmax**:

$$p_i = \frac{e^{z_i}}{\sum_{j \in V} e^{z_j}}$$

- Each $p_i \in (0, 1)$ and $\sum_i p_i = 1$ — a valid distribution.
- Exponentiation makes bigger logits dominate; it's monotonic, so ranking is preserved.

Temperature sharpening vs flattening

- Temperature T rescales the logits **before** softmax:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- $T \to 0$: the largest logit dominates → distribution collapses to **argmax** (greedy, deterministic).
- $T = 1$: the model's "natural" distribution.
- $T \to \infty$: logits flatten → **uniform** (maximum randomness).

Temperature sharpening vs flattening

- Temperature (T) changes the logits **before softmax**.
- It **controls** how predictable or varied the output will be.
  - **Low temperature ((T<1))**: *probability differences become larger*. The highest-scoring token dominates.
  - **(T=1)**: uses the model's original probability distribution.
  - **High temperature ((T>1))**: *probability differences become smaller*. More tokens have a chance of being selected.
  - **Argmax** means selecting the token with the highest score.
  - **Uniform** means tokens become almost equally likely.

Example: "The chicken crossed the…"

Low (T): almost always selects **"road." |** Normal (T): probably selects **"road." |** High (T): may select **"street," "river,"etc**

Temperature sharpening vs flattening

Sharpening: Makes the highest-probability token much more dominant. The output becomes predictable.

Flattening: Distributes probability more evenly among tokens. The output becomes varied or creative.

Distribution	Road	Street	River
Original	70%	20%	10%
Sharpened	90%	8%	2%
Flattened	45%	32%	23%

Low temperature → sharpening

High temperature → flattening

Let's go for a quick playthrough

1. Temperature (top-p pinned at 1.0)

```
for t in (0.0, 1.0, 2.0):
    ask(t, 1.0)
```

[22] ✓ 47.4s

```
temperature=0.0  top_p=1.0
    Guiding ships home, he stands alone.
    Guiding ships home, he stands alone.
    Guiding ships home, he stands alone.

temperature=1.0  top_p=1.0
    Light shines bright; they sail home.
    Light guides ships; I remain alone.
    Lonely light keeps ships from rocks.

temperature=2.0  top_p=1.0
    Light beacons cut through dark loneliness.
    Lost sailor signals home, beams died.
    Silent stone lantern guides lost souls.
```

What is Instruction Tuning?

01

Definition

A technique for fine-tuning Large Language Models (LLMs) using a labeled dataset of instructional prompts and their corresponding desired outputs.

02

The Goal

It teaches a pre-trained model how to follow specific user instructions rather than just generating linguistically coherent text.

03

Practical Impact

It is the critical step that transforms a basic text-generator into a useful, interactive assistant (e.g., turning a base model into ChatGPT).

Why Do We Need It?

01

The Problem with Base Models

Pre-trained models are optimized only to predict the next word.

Example: If you prompt "Teach me how to bake bread," it might just append "in a home oven" rather than providing a recipe.

02

Bridging the Gap

Instruction tuning aligns the model's fundamental objective (next-word prediction) with the user's actual intent.

03

Efficiency

It requires a fraction of the computational power, time, and data compared to training an massive model from scratch.

How It Works & Training Data

01

The Mechanism

Models learn from Supervised Fine-Tuning (SFT) using datasets with three elements:

- **Instruction: The task (e.g. "Translate")**
- **Context: Optional info**
- **Target: Correct response**

02

Human-Created

High-quality but labor-intensive datasets curated by humans.

Examples: Flan, Dolly, OpenAssistant

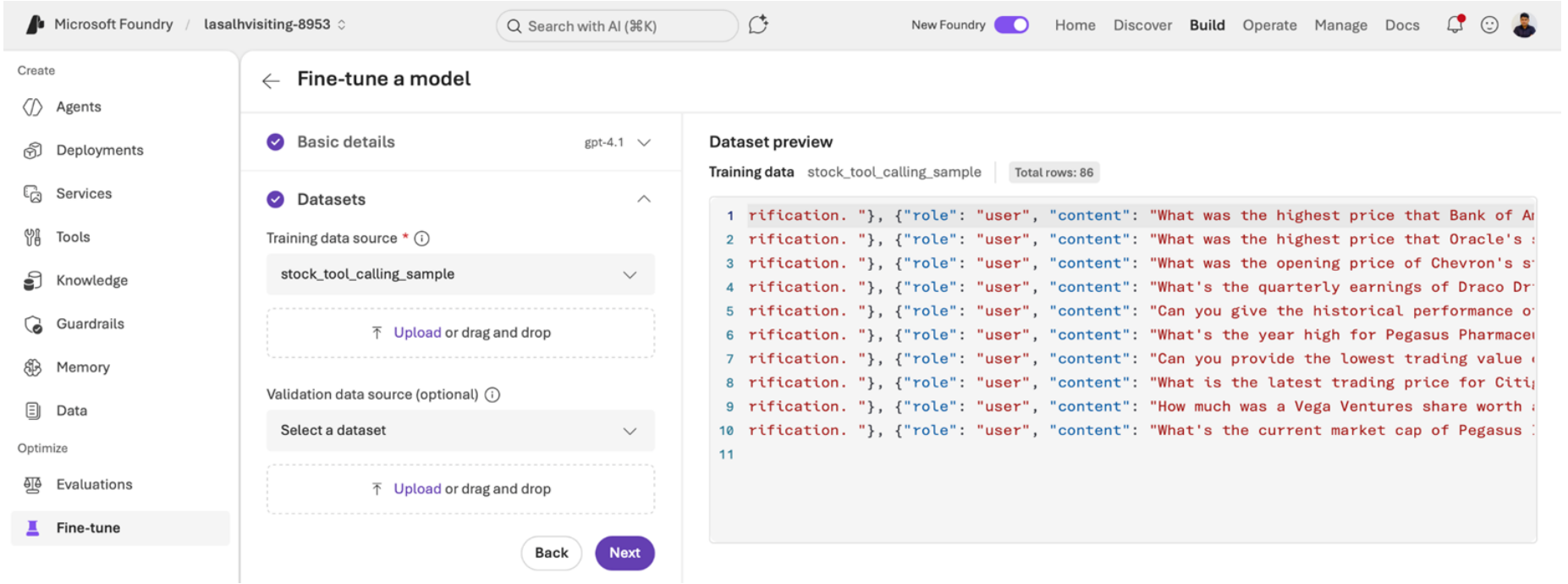
03

LLM-Generated

Using outputs from larger models (like GPT-4) to cheaply train smaller models.

Examples: Alpaca, ShareGPT

Quick Demo


Microsoft Foundry / lasalhvisiting-8953
Search with AI (⌘K)
New Foundry
Home
Discover
Build
Operate
Manage
Docs
Create
Agents
Deployments
Services
Tools
Knowledge
Guardrails
Memory
Data
Optimize
Evaluations
Fine-tune
Fine-tune a model
Basic details
gpt-4.1
Datasets
Training data source *
stock_tool_calling_sample
Upload or drag and drop
Validation data source (optional)
Select a dataset
Upload or drag and drop
Back
Next
Dataset preview
Training data
stock_tool_calling_sample
Total rows: 86
1 rification. "}, {"role": "user", "content": "What was the highest price that Bank of A
2 rification. "}, {"role": "user", "content": "What was the highest price that Oracle's
3 rification. "}, {"role": "user", "content": "What was the opening price of Chevron's s
4 rification. "}, {"role": "user", "content": "What's the quarterly earnings of Draco Dr
5 rification. "}, {"role": "user", "content": "Can you give the historical performance o
6 rification. "}, {"role": "user", "content": "What's the year high for Pegasus Pharmace
7 rification. "}, {"role": "user", "content": "Can you provide the lowest trading value
8 rification. "}, {"role": "user", "content": "What is the latest trading price for Citi
9 rification. "}, {"role": "user", "content": "How much was a Vega Ventures share worth
10 rification. "}, {"role": "user", "content": "What's the current market cap of Pegasus
11

The three defining limitations

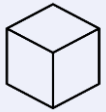
- There are 3 defining limitations when it comes to using models alone
- Almost everything we build this term compensates for these:

Limitation	What happens there	
Stateless	No memory between calls	If you want the model to "remember," you must store and replay it.
Ungrounded	No live data; only training + your input	To answer about your documents or current facts, you must bring the data to the model (retrieval) or give it tools to fetch it.
Probabilistic	Improvises; "best" not guaranteed	We control this with the **temperature**

Evolution from standalone models to Agentic AI

AI that can

Create for you

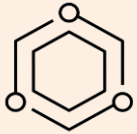


Models

- Text summarization
- Text synthesis
- Pattern matching

AI that can

Retrieve for you



Assistants

- Information retrieval
- Prescriptive tasks
- Single step or rule based processes

AI that can

Do for you



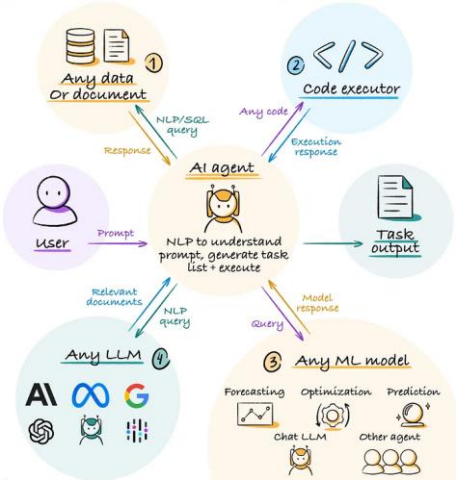
Agents

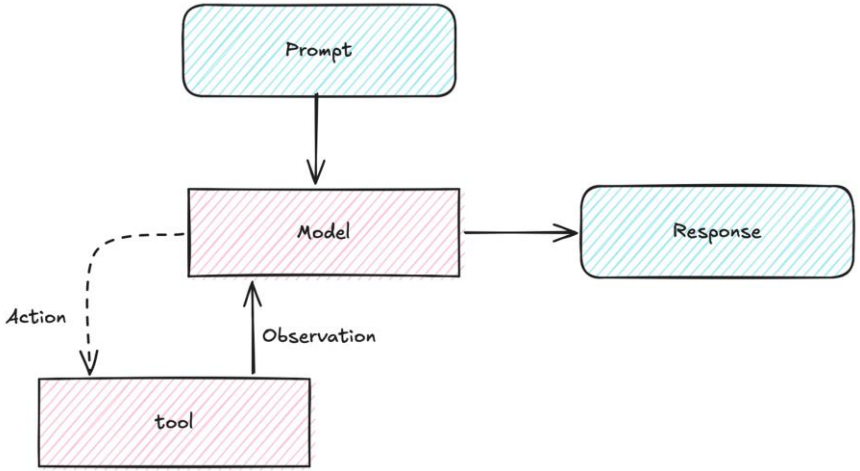
- Multi step processes
- Autonomous actions
- Self-corrections

- The evolution from *probabilistic* to *agentic* and how we overcome the limitations of LLMs
- A **model** gives an answer, an **assistant** finds information, but an **agent** completes a task.

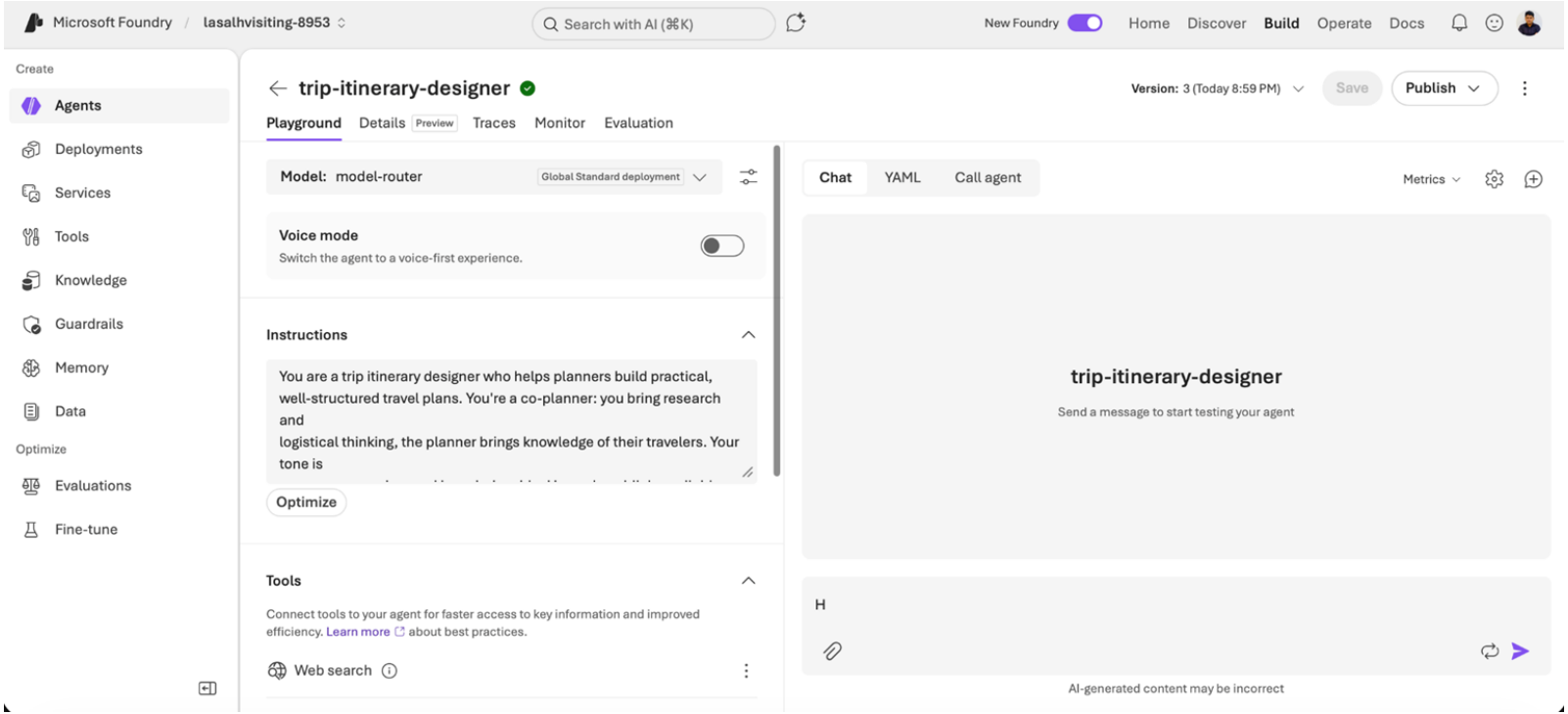
What is an Agent?

Definition: An LLM directing a loop: it observes state, decides on actions (tool calls), a runtime executes them, and results feedback.





Quick demo on playground



The Agency Spectrum

Definition: Agency means how much control the AI has over completing a task.

Level	Who controls the process?	Example
LLM call	The user provides one task; AI creates one response.	Summarize this email
Workflow / chain	The developer fixes the sequence of steps. The model follows this predefined path.	Translate → summarize → format
Router	The model selects one path from several predefined paths.	Send a customer request to billing support or technical support.
Autonomous agent	The model decides which tools to use, their order and when to stop.	Book me a flight to London tomorrow It should request human approval before making the payment.

Do You Actually Need an Agent?

- Agents provide flexibility and but they can have: latency, cost, and non-determinism
- Every LLM decision is a chance to be wrong

Ask yourself…	Then reach for…
Can we define all the steps in advance?	Use a fixed **workflow**. *Example: Validate a form → save it → send a confirmation email.*
Are there only a few known outcomes?	Use a **router** or **classifier**. *Example: Route a request to Billing, Sales or Technical Support.*
Does the task require changing multi-step decisions?	Use an **agent** with only the required level of autonomy. *Example: Create a travel plan by checking weather, budget and availability.*

Always select the simplest and most reliable solution that can complete the task.

Checkpoint 1 – Justify which pattern you would use



1. You are building an application that takes a daily news article in Spanish, translates it into English, summarizes it into three bullet points, and finally formats the output into a specific JSON structure for your database.

2. A user prompts your application with: "Find me a flight to London tomorrow, book a 4-star hotel near the city center, and check if I need an umbrella based on the forecast." The system must interact with external booking APIs and weather services to fulfill the request.

3. Your company receives thousands of customer emails daily. You need a system that reads each incoming email and simply categorizes it into one of three buckets: "Billing Issue," "Technical Support," or "General Inquiry."

Pair up — 3 minutes — then we discuss answers together.

Checkpoint 1 – Answers

1. Workflow / Chain

You can write the steps down in advance (Translate → Summarize → Format). It relies on a fixed code path with specific LLM steps rather than autonomous decision-making.

2. Autonomous Agent

This requires dynamic multi-step decisions against an unpredictable environment (flight/hotel availability). The LLM must observe state, decide which tools to use and in what order, and execute them.

3. Router / Classifier

The output space is small and known (only three categories). The LLM just needs to decide which of the N fixed paths the input belongs to.

Rule of thumb — choose the least autonomy that still solves the problem.

PART 2 OF 4

ReAct & Tool Calling

- ReAct tool calling
- Tool execution lifecycle



ReAct: why interleaving beats planning ahead

PROMPT: "Who is the current CEO of the company that owns LinkedIn, and what year did they become CEO?":

- **Plan-then-execute**: write the full plan, run it. Fragile — the plan can't react to what search actually returns (what if the owner changed? what if search returns a disambiguation page?).
- **ReAct (Reasoning + Acting):** alternate Thought → Action → Observation, deciding each action after seeing the last observation:

```
Thought: I need the owner of LinkedIn first.          | reasoning (tokens)
Action:  search("who owns LinkedIn")                  | tool call
Observation: "Microsoft acquired LinkedIn in 2016"    | runtime result
Thought: Now I need Microsoft's current CEO…          | reasoning
Action:  search("Microsoft current CEO start year")   | tool call
Observation: "Satya Nadella, CEO since 2014"          | result
Thought: I can answer now.
Answer:  Satya Nadella, CEO since 2014.
```

The Messages

Definition: Messages are the fundamental unit of context for models in LangChain. They represent the input and output of models, carrying both the content and metadata needed to represent the state of a conversation when interacting with an LLM..

Messages are objects that contain:

- **Role** - Identifies the message type (e.g. `system`, `user`)
- **Content** - Represents the actual content of the message (like text, images, audio, documents, etc.)
- **Metadata** - Optional fields such as response information, message IDs, and token usage

Message types

- System message - Tells the model how to behave and provide context for interactions
- Human message - Represents user input and interactions with the model
- AI message - Responses generated by the model, including text content, tool calls, and metadata
- Tool message - Represents the outputs of tool calls

https://docs.langchain.com/oss/python/langchain/messages
https://developers.openai.com/api/docs/guides/function-calling

The Bridge problem

The problem: LLMs produce text

Software needs structured commands. **Parsing** free-form text or model-written JSON can fail because of extra text, invalid syntax, missing fields, or wrong data types.

The solution: Native tool calling

The model returns a structured request such as

get_weather({"city":"Kandy"}).

The application validates the tool name, inputs, permissions, and business rules before execution.

User request → LLM → Structured tool call → Validation → Tool → Result → Final answer

The LLM requests the action; the application validates and executes it.

Frame by Frame: Tool calling cycle



- **Tools** extend what agents can do letting them fetch real time data, execute code, query external databases, and take actions in the world.
- Under the hood, tools are callable functions with well defined inputs and outputs that get passed to a chat model. The model decides when to invoke a tool based on the conversation context, and what input arguments to provide.

```python
from langchain.tools import tool

@tool
def search_database(query: str, limit: int = 10) -> str:
    """Search the customer database for records matching the query.

    Args:
        query: Search terms to look for
        limit: Maximum number of results to return
    """
    return f"Found {limit} results for '{query}'"
```

Checkpoint 2 – Quick Activity

1. The Scenario: Brainstorm one specific real-world task where an AI agent cannot rely on its internal knowledge and must interact with an external system (e.g., checking live sports scores, sending an email, booking a meeting room).

2. Design the Signature: Give your tool a clear function name and list the exact input parameters (arguments) the LLM would need to extract from a user's prompt (e.g., book_room(room_id, start_time, duration)).

3. Determine the Return: What specific data type or confirmation must this tool return back to the LLM so it can formulate a helpful final response to the user?

Pair up — 3 minutes — then we discuss answers together.

PART 3 OF 4

Agentic Frameworks

- Agentic frameworks
- How to choose the correct framework



Agentic frameworks

Definition: An agentic framework is a software architecture designed to build, manage, and orchestrate autonomous AI agents.

Overcoming the "Stateless" Limitation

A raw LLM cannot pause a task, wait for an external system to process a job, and come back to it later. Agentic frameworks manage the "state" of a task. If a process takes hours or requires waiting for a human to approve an action, the framework parks the state and resumes it seamlessly.

Deterministic Control in a Probabilistic System

LLMs are probabilistic. Frameworks enforce structure. They ensure that if an LLM decides to delete a file, the framework intercepts that decision, validates the parameters, and can enforce a "human-in-the-loop" approval process before the action actually happens.

Cyclical Execution and Error Correction

The agent can execute a tool (like running code), read the error message, realize its mistake, and try a different approach without the user having to re-prompt it.

Multi-Agent Orchestration

Complex problems often require different skill sets. Instead of using one massive prompt to do everything, frameworks allow developers to build multi-agent systems.

Key Components of an Agentic Framework

Models

The brain of the system. While the LLM is the most famous part, an agentic framework often uses a mix of models.

Tools

Tools give the LLM "hands" to interact with the outside world. The framework translates the LLM's text output into functional calls.

Memory

Memory provides the context necessary for the agent to maintain continuity across tasks and sessions.

Monitoring

This is the monitoring backend for the agents execution.

Prompts

The instructions and the response objects that govern the HUMAN, SYSTEM, TOOL, AI lifecycle

Middleware / Orchestration

These include the structured outputs, guardrails, HIL, Orchestration etc.

The prompt guides the model, memory provides context, tools perform actions, orchestration controls the process, and monitoring records everything.

Popular Agentic Frameworks

Framework	Best at	Reach for it when…
LangChain + LangGraph *(our stack)*	biggest ecosystem of integrations; LangGraph: explicit, durable, resumable control flow	you want maximum transferability and production-grade control
Microsoft Agent Framework (succeeds AutoGen + Semantic Kernel)	multi-agent patterns with first class citizenship	deep Azure ecosystem alignment
LlamaIndex	data first: ingestion, indexing, retrieval quality knobs	RAG is the product and agents are a feature (inverse of LangChain's emphasis)
Google ADK	thorough multi agent orchestration with sequential pipelines, parallel workflows and feedback loops	quick prototyping and deep orchestration flows and handoffs
DSPy	*optimizing* prompts/pipelines programmatically (adjacent to agents)	you have evals and want the machine to tune the prompts

Popular agentic frameworks

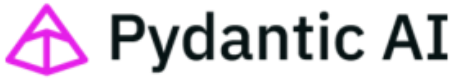


















Best Practices

- Start with Small-Scale Prototypes
- Incorporate Human Oversight
- Optimize Memory Usage
- Ensure Robust Monitoring and Observability
- Ensure security is not an afterthought

https://blog.jetbrains.com/pycharm/2026/06/top-agentic-frameworks-for-building-applications-2026/#best-agentic-frameworks-for-your-projects
https://www.exabeam.com/explainers/agentic-ai/agentic-ai-frameworks-key-components-top-8-options/

Next Week & How to Prepare

Lecture 06 — Knowledge & memory of AI agents: RAG, embeddings, retrieval, memory & the idea of tools

- Go through the LABs
  - Go through background material on the above topics
  - Play around on Azure AI Foundry